CodeRoom:

A Web-Based Automated Code Evaluation and Learning Platform

Minor Project II

Submitted By:

Hansi Gupta (23803019)

Yash Gupta (23103007)

Kanishk Richhariya (23103125)

Under the supervision of – **Mr. Amitesh**
Co-Supervisor – **Prof. Manish Kumar Thakur**



May – 2026

Submitted in partial fulfilment of the Degree of

Bachelor of Technology / Integrated Master of Technology

Computer Science Engineering

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING AND INFORMATION TECHNOLOGY

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA

TABLE OF CONTENT

Declaration 3

Self-Declaration 4-5

Certificate 6

Acknowledgement 7

Summary 8

Chapter-1 Introduction 9-16

1.1 General Introduction

1.2 Problem Statement

1.3 Significance/Novelty of the problem

1.4 Empirical Study (Existing Tool Survey & Experimental Study)

1.5 Brief Description of the Solution Approach

1.6 Comparison of existing approaches to the problem framed

Chapter-2 Literature Survey 17-19

2.1 Summary of papers studied

2.2 Integrated summary of the literature studied

Chapter-3 Requirement Analysis and Solution Approach 20-23

3.1 Overall description of the project

3.2 Requirement Analysis

3.5 Solution Approach

Chapter-4 Modelling and Implementation Detail 24-40

4.1 Design Diagrams

4.1.1 Control Flow Diagrams

4.1.2 Work Flow Diagrams

4.1.3 Use Case Diagrams

4.1.4 Class Diagrams

4.1.3 Sequence Diagrams

4.2 Implementation Details

Chapter-5 Testing 41-50

5.1 Testing Plan

5.2 Component Decomposition and Type Of Testing Required

5.3 List Of Test Cases

5.4 Error and Exceptional Handling

5.5 Test Verification Certificate

Chapter-6 Findings, Conclusion, and Future Work 51-56

6.1 Findings

6.2 Comparison

6.3 Conclusion

6.4 Future Work

References IEEE Format (Listed alphabetically) 57

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Place: Jaypee Institute of Information Technology, Noida

Date: 05 May 2026

Enrolment Number	Name of Student	Signature
23803019	Hansi Gupta	
23103007	Yash Gupta	
23103125	Kanishk Richhariya	

STUDENTS' SELF DECLARATION FOR OPEN-SOURCE LIBRARIES AND OTHER SOURCE CODE USAGE IN MINOR PROJECT

We**, Hansi Gupta, Yash Gupta, Kanishk Richhariya,** hereby declare the following usage of the code and prebuilt libraries in our minor project in **6th** Semester with the consent of our supervisor. We also measure the similarity percentage of pre written source code and our source code and the same is mentioned below. This measurement is true with best of our knowledge and abilities.

1. List of pre build libraries (On next page)

2. List of pre build features in libraries or in source code. (On next page)

3. Percentage of pre written source code and source written by us. 25%-75%

Enrolment Number	Name of Student	Signature
23803019	Hansi Gupta	
23103125	Yash Gupta	
22103007	Kanishk Richhariya	

Declaration by Supervisor (To be filled by Supervisor only)

I, **Mr Amitesh** (Name of Supervisor) declares that I above submitted project with Titled "**CodeRoom: A Web-Based Automated Code Evaluation and Learning Platform"** was conducted in my supervision. The project is original and neither the project was copied from External sources nor it was submitted earlier in JIIT. I authenticate this project.

(Any Remarks by Supervisor)

Signature (Supervisor)

1. List of Pre-Built Libraries

We, Yash Gupta, Hansi Gupta, and Kanishk Richhariya, hereby declare the following usage of open-source code and pre-built libraries in our Minor Project in 6th Semester with the consent of our supervisor.

- **Judge0 API:** For secure and sandboxed online code execution.
- **Express.js:** To handle routing and API endpoints in the backend.
- **MongoDB:** For data persistence of users, problems, submissions, and results.
- **React.js & TailwindCSS:** For responsive frontend design and interaction.
- **JWT Authentication:** For secure session management and login.
- **Docker :** For containerizing and testing isolated executions.

2. List of Pre-Built Features in Libraries or Source Code

The following features were utilized directly from the libraries or tools:

- Online compilation using Judge0 sandbox.
- REST API for submission handling and real-time result fetching.
- Token-based authentication for secure access.
- MongoDB aggregation for analytics and dashboards.

CERTIFICATE

This is to certify that the work titled "**CodeRoom: A Web-Based Automated Code Evaluation and Learning Platform**" submitted by "**Hansi Gupta, Yash Gupta, Kanishk Richhariya**" in partial fulfilment for the award of degree of "**B.Tech/ INTGT M.Tech Computer Science Engineering**" of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of this or any other degree or diploma.

Signature of Supervisor: ……

Name of Supervisor: ……

Designation: ……

Date: ……

ACKNOWLEDGEMENT

We express my sincere gratitude to **Mr. Amitesh**, Assistant Professor (Senior Grade), Dept. of CSE/IT, India, for her stimulating guidance, continuous encouragement and supervision throughout the course of present work.

We would like to place on record our deep sense of gratitude to our mentor **Mr. Rajat**, 4th year, Jaypee Institute of Information Technology, India for his generous guidance, help and useful suggestions.

We also wish to extend our thanks to our respective families and friends for their insightful comments and constructive suggestions to improve the quality of this project work. And we also thank God, who did not let us down and helped us completing this project by keeping up our moral and dedication towards this project.

Date: **05 may 2026**

Enrolment Number	**Name of Student**	**Signature**
23803019	Hansi Gupta	
23103007	Yash Gupta	
22103125	Kanishk Richhariya	

SUMMARY

The project CodeRoom: A Web-Based Automated Code Evaluation and Learning Platform is an innovative, academic-focused system designed to revolutionize the process of coding assignment management, submission, and evaluation within educational institutions. The primary aim of CodeRoom is to simplify, automate, and standardize the way programming exercises are assigned, executed, and graded—thereby creating a seamless bridge between instructors and students in computer science education.

In the conventional academic setting, coding assignments are often evaluated manually, which is both time-intensive and error-prone. Faculty members are required to download each student's submission, test it across multiple input cases, and manually assign scores. This process not only delays feedback but also restricts real-time learning opportunities. To address these challenges, CodeRoom introduces an automated and evaluation ecosystem that allows professors to efficiently create, manage, and grade assignments, while enabling students to learn and experiment interactively.

The platform integrates the following components:

- **Frontend:** Interactive coding interface built with React.js.
- **Backend:** Node.js + Express REST APIs for assignment management and evaluation.
- **Database:** MongoDB for storing user and submission data.
- **Execution Layer:** Judge0 API for secure, containerized code execution.

This system aims to overcome the inefficiencies of manual evaluation, reduce faculty workload, and promote faster feedback loops. With scalability, concurrent submission handling, and analytics integration, CodeRoom provides an end-to-end academic coding management solution.

Chapter – 1

INTRODUCTION

1.1 General Introduction

The project CodeRoom: A Web-Based Automated Code Evaluation and Learning Platform focuses on simplifying and automating the process of coding assignment creation, submission, and evaluation within academic environments.The system provides a unified platform where professors can create programming assignments, define sample, hidden, and random test cases, and set deadlines. Students can write, compile, and submit their code online using an integrated editor without any need for local setup or compiler installation. Each submission is automatically executed and evaluated against the predefined test cases in a secure, isolated environment using the Judge0 API.The platform architecture consists of a React.js and TailwindCSS frontend, a Node.js and Express.js backend, and a MongoDB database. Professors interact through a clean and intuitive dashboard to manage problems and track submissions, while students have a distraction-free coding interface. The backend handles user authentication, manages assignments, and communicates with the execution API to fetch results. The database stores user details, submissions, problem statements, and performance analytics efficiently.

CodeRoom significantly reduces manual workload by automating evaluation, ensuring instant feedback and consistent grading for all students. It supports multiple concurrent submissions, making it scalable for classroom and lab environments. The system also maintains fairness and transparency by using standardized test cases and performance metrics.Overall, CodeRoom enhances the teaching and learning experience by providing a faster, more reliable, and interactive way to assess coding skills. It serves as a bridge between traditional classroom evaluation methods and modern, data-driven learning systems.

1.2 Problem Statement

Managing code assignments manually presents several challenges:

- Time-consuming manual checking of multiple submissions.
- Lack of uniform test case validation across students.
- Delayed feedback loop affecting student learning.
- Limited scalability during lab sessions with many concurrent users.
- Inability to provide real-time compilation or debugging assistance.

Existing platforms either lack integration with academic workflows or are overly complex for classroom settings. There is a need for a lightweight, secure, and customizable system that automates the entire lifecycle of code submission, evaluation, and result generation.

1.3 Significance/Novelty of the Project

The CodeRoom project introduces a modern, efficient, and educationally tailored approach to automating code evaluation within academic environments. Its novelty lies in combining automation, real-time execution, and analytical insights within a single, user-friendly web platform designed specifically for universities and institutions.

1. Integrated Compiler:

   CodeRoom integrates the Judge0 API to execute code within sandboxed Docker containers. This ensures that every code submission is executed securely and independently, preventing unauthorized system access or resource misuse. The cloud-based architecture eliminates the need for local compilers, providing students with a consistent development environment accessible from any device.

2. Customizable Test Framework:

   The platform supports multiple types of test cases sample, hidden, and randomized allowing instructors to create dynamic and fair evaluations. Sample test cases guide students in understanding problem requirements, while hidden and random test cases ensure that only correct, efficient solutions receive full credit, discouraging hardcoded or partial solutions.

3. Real-time Performance Analytics:

   CodeRoom provides an integrated analytics dashboard for instructors to monitor student performance trends, such as average accuracy, most common runtime errors, and submission frequency. This data-driven approach helps educators identify weak areas in understanding and improve teaching strategies accordingly.

4. Institutional Customization:

   Unlike generic competitive coding platforms, CodeRoom is built for academic adaptability. Professors can create private assignments, set submission deadlines, adjust scoring parameters, and even export detailed reports for evaluation. This makes it ideal for integration into university-level course management and internal assessments.

5. Multi-language Execution:

   The platform supports multiple programming languages including C, C++, Python, and Java, ensuring flexibility for varied course syllabi. This multilingual support prepares students for real-world programming diversity and enables departments to conduct cross-language evaluations easily.

Overall, the novelty of CodeRoom lies in its ability to combine secure cloud-based execution, automated evaluation, and educational analytics into a cohesive, scalable system. By bridging the gap between traditional manual grading and modern digital automation, it enhances both teaching efficiency and student learning outcomes in programming education.

1.4 Empirical Study

1.4.1 Challenges in Existing Code Evaluation Platforms

With the rapid digitalization of education, many online coding and evaluation platforms such as HackerRank, CodeChef, and LeetCode have gained widespread popularity among students and professionals. However, despite their extensive reach, these platforms are not always suitable for academic or institutional environments.Instructors often face challenges in adapting such systems to classroom requirements, as most platforms are competition-oriented and lack features like assignment deadlines, classroom dashboards, and internal result

analytics. Moreover, students face difficulties accessing uniform environments due to local compiler dependencies, inconsistent grading methods, and the absence of real-time academic feedback.These shortcomings create a clear need for a dedicated system that focuses on secure, scalable, and education-specific code evaluation while providing a simplified experience for both students and teachers.

1.4.2 Limitations of Current Code Evaluation Systems

A detailed analysis of existing online judges and classroom coding environments revealed significant limitations. Platforms like HackerRank and HackerEarth rely on centralized competitive models, where users solve challenges in a shared environment without individualized instructor control. Similarly, Moodle with the VPL Plugin offers code evaluation inside a learning management system, but its user interface is dated and lacks scalability for concurrent submissions.Most current systems fail to support custom test case management, real-time performance analytics, and automatic grading integration for specific classroom requirements. Additionally, limited institutional control, restricted reporting capabilities, and a lack of fine-grained feedback make it difficult for educators to monitor student progress effectively. Hence, there is a gap between existing general-purpose coding platforms and the specialized requirements of academic coding assessments.

1.4.3 Dataset Insights and Learner Behaviour

To further understand user behaviour and evaluation needs, we studied institutional coding workflows and conducted small-scale testing using sample datasets of student submissions. The observations revealed common behavioural patterns such as frequent syntax errors, logic mistakes, and compilation issues, highlighting the importance of real-time feedback and error explanation.We also noted that students preferred immediate execution results rather than delayed grading, which supports the need for automated evaluation and instant feedback. Professors, on the other hand, emphasized the necessity for plagiarism detection, analytics visualization, and easy download of grade reports, all of which are largely missing in existing tools. These insights directly influenced the architecture and feature design of CodeRoom, focusing on automation, fairness, and accessibility.

1.4.5 Bridging the Gap with a Proposed Solution

From this empirical study, it becomes evident that no single existing platform fulfills all the requirements of an academic evaluation system. Competitive programming platforms focus on open challenges, while learning management tools lack robust code execution capabilities.The proposed CodeRoom system bridges this gap by combining the strengths of both approaches offering real-time compilation, multi-language support, instructor control, and automatic grading, all integrated into a user-friendly academic framework.By emphasizing institutional customization, real-time feedback, and analytics, CodeRoom creates a hybrid academic evaluation ecosystem that addresses both scalability and educational usability.

1.5 Brief Description of the Solution Approach

Our project CodeRoom: A Web-Based Automated Code Evaluation and Learning Platform integrates multiple modern technologies to create a seamless, secure, and efficient environment for code submission and evaluation. The system architecture is composed of four major components that work together to automate the entire process—from problem creation to code execution and feedback delivery.

Frontend:

The frontend of the system is developed using React.js and TailwindCSS, providing an intuitive and responsive user interface for both students and instructors. The student dashboard includes an integrated online code editor that allows users to write, compile, and submit programs in real time. The professor dashboard enables assignment creation, test-case management, and result visualization. The interface focuses on simplicity, clarity, and minimal latency to ensure a smooth experience during live coding sessions or lab evaluations.

Backend:

The backend is built using Node.js with the Express.js framework. It handles all core application logic, such as user authentication, problem management, and submission routing. It ensures secure login through JWT-based authentication and efficiently manages multiple concurrent submissions using asynchronous request handling. This layer also validates input

data and maintains synchronization between all connected modules.

Execution Layer:

The execution layer is powered by the Judge0 API, which provides a containerized and sandboxed environment for compiling and executing user code. Each submission is executed securely in an isolated Docker container, preventing any unauthorized access or interference with the system. Judge0 supports multiple programming languages such as C, C++, Python, and Java, enabling flexibility for different course requirements. The results, including standard output, runtime errors, and execution time, are fetched back through REST calls to ensure accurate and real-time evaluation.

Database:

The system uses MongoDB as the primary database for storing user information, assignment details, test cases, and submission records. Its document-based structure allows flexible schema design and fast data retrieval. The database also maintains historical submission data to support analytics and performance tracking for both students and instructors.

Data Flow:

The overall data flow begins when a student submits code through the frontend interface. The submission request is transmitted to the backend API, which forwards it to the Judge0 API for execution. The evaluated result is then returned to the backend, stored in MongoDB, and displayed instantly on the student’s dashboard. This pipeline ensures real-time feedback, secure execution, and accurate record-keeping.

1.6 Comparison of Existing Approaches to the Problem Framed

Table 1.1-CodeRoom VS Other Platforms

Feature / Approach	CodeRoom	Geeks for Geeks	CodeChef	CodeForces	Moodle VPL	Google Classroom
Purpose	Academic+ Competitive Hybrid	Learning + Practice	Competitive Coding	Competitive Coding	Academic LMS	Assignment Distribution and Submission
Assignment Creation and Distribution	✅ Yes	❌ No	❌ No	❌ No	⚠ Partial	✅ Yes but no coding evaluation
Instant Feedback to Students	✅ Instant	✅ Yes	✅ Yes	✅ Yes	❌ No	❌No(teacher reviews manually)
Automatic Code Evaluation	✅ Full test case control	✅ Contest only	✅ Contest only	✅ Contest only	⚠ Limited	❌ No (only file upload)
Ease of Use (UI/UX)	✅ Minimal & Responsive	✅ Responsive	✅ Yes	⚠Technical Layout	⚠Outdated	✅ Yes
Teacher Analytics and Reports	⚠ Basic Charts	❌ No	❌ No	❌ No	⚠Basic Reports	⚠ Basic(turn in stats only)
Institution Level Scalability	✅ Yes	❌ No	❌ No	❌ No	❌ No	✅ Yes (already scaled platform)
Plagiarism/ Similarity Checking	❌ No(Future Enhancements)	❌ No	⚠ Delayed	❌ No	❌ No	⚠ Limited
Works Without Internet	❌ No	❌ No	❌ No	❌ No	❌ No	❌ No

- **GeeksforGeeks**
    - **Problem:** No assignment or academic evaluation system.
    - **CodeRoom Fix:** Provides full assignment creation and auto-evaluation for classrooms.

- **CodeChef/CodeForces**
    - **Problem:** Only supports contests, not structured classroom workflows.
    - **CodeRoom Fix:** Enables instructor-controlled assignments with deadlines and test-case management.

- **Moodle VPL**
    - **Problem:** Outdated UI and slow execution with limited concurrency.
    - **CodeRoom Fix:** Provides a modern UI with fast, scalable execution powered by Judge0.

- **Google Classroom**
    - **Problem:** Cannot compile or auto-evaluate code (only handles file uploads).
    - **CodeRoom Fix:** Integrates real-time code execution and instant automated scoring.

- **Manual Evaluation**
    - **Problem:** Very time-consuming and inconsistent across large batches.
    - **CodeRoom Fix:** Automates compiling, testing, grading, and reporting within seconds.

Chapter – 2

LITERATURE SURVEY

2.1 Summary of papers studied

- **Scaling Automated Programming Assessment Systems**

Author: I. Mekterović

Published in: *Electronics (MDPI)*, 2023

Link: https://www.mdpi.com/2079-9292/12/4/942

This peer-reviewed paper presents a detailed examination of modern Automated Programming Assessment Systems (APAS). It focuses on scalability, container-based sandboxing, and detection of cheating or code similarity. The study emphasizes that manual programming assessment becomes infeasible for large classes and recommends secure, isolated execution of untrusted code. These findings directly support CodeRoom's choice of a sandbox-based execution model, ensuring fairness and consistency across multiple concurrent submissions.

- **Online Judge System: Requirements, Architecture, and Trends**

Author: Y. Watanobe

Published in: International Journal of Software Engineering and Knowledge Engineering *(World Scientific)*, 2022

Link: https://www.worldscientific.com/doi/pdf/10.1142/S0218194022500346

This work provides a comprehensive architectural overview of Online Judge (OJ) systems, including functional requirements (test-case design, language support, feedback mechanisms) and non-functional needs (security, reliability, scalability). The paper highlights integration challenges between OJ systems and academic Learning Management Systems (LMS). These insights inform CodeRoom's design, especially its separation between academic workflow management and code execution services.

- **Learning Analytics in Programming Courses: Review and Implications**

  Authors: U. Omer, R. Tehseen, M. Farooq, A. Abid
  Published in: *Education and Information Technologies (Springer)*, 2023
  Link: https://link.springer.com/article/10.1007/s10639-023-11611-0

  This systematic review explores the application of learning analytics (LA) to programming education. It identifies analytics patterns such as submission frequency, runtime error trends, and time-to-solve metrics as effective indicators for predicting learner performance. The findings inspired CodeRoom's analytics module, enabling instructors to visualize student progress

2.2 Integrated summary of the literature studied

Table 2.1-Summarization of Research Papers

Paper	Publisher	Focus Area	Key Findings	Relevance to CodeRoom
Mekterović (2023) –Scaling Automated Programming Assessment Systems	MDPI – Electronics	Automated programming assessment, scalability, sandboxing	Highlights need for secure, scalable, automated assessment systems	Supports CodeRoom's choice of container-based, automated execution
Watanobe (2022) – Online Judge System: Requirements, Architecture, and Trends	World Scientific	Online Judge architecture, functional + non-functional requirements	Define OJ structure, LMS integration challenges, security requirements	Guides CodeRoom's architectural design & test-case workflow
Omer et al. (2023) – Learning Analytics in Programming Courses	Springer	Learning analytics, student performance tracking	Shows analytics improve engagement, identify weak learners	Forms basis for CodeRoom's analytics dashboard

Chapter – 3

REQUIREMENT ANALYSIS AND SOLUTION APPROACH

3.1 Overall description of the Project

The project CodeRoom is a comprehensive, web-based, multi-user platform designed to automate the process of coding assignment evaluation in academic settings. It provides a centralized environment where both instructors and students can interact seamlessly through an intuitive and responsive interface.The platform enables professors to create assignments, upload problem statements, define test cases, and monitor submissions, while students can write, compile, and submit their code directly through the integrated web editor. By combining automation with analytics, CodeRoom eliminates the challenges of manual grading and improves the efficiency, accuracy, and transparency of the entire evaluation process.

3.2 Requirement Analysis

3.2.1 Functional Requirements: Creation of User Dataset

1. **User Management:**

   The system supports multiple user roles — primarily Professors and Students — each with specific permissions and functionalities. Professors can create, modify, and evaluate assignments, while students can attempt problems, submit solutions, and view results in real time.

2. **Assignment Creation:**

   Professors can easily create programming problems with detailed descriptions and define sample, hidden, and random test cases. This helps ensure fairness and prevents hard-coded solutions, maintaining the integrity of the evaluation.

3. **Submission and Execution:**

   Students can write code directly in the online editor and execute it instantly using the integrated compiler. The system supports multiple programming languages and

provides instant output and error feedback, improving the learning experience.

4. **Automatic Evaluation:**

   Each code submission is automatically compiled and tested against predefined test cases using the Judge0 API. The backend compares outputs, assigns marks, and categorizes results as “Accepted”, “Wrong Answer”, or “Compilation Error”.

5. **Reports and Analytics:**

   The system generates detailed performance reports for instructors, including metrics like submission count, average accuracy, and frequently occurring errors. Graphical analytics help professors identify patterns in student performance and optimize their teaching strategies.

6. **Concurrent Execution:**

   The platform is designed to handle multiple submissions simultaneously, ensuring scalability during lab exams or assignment deadlines. The use of asynchronous processing allows smooth execution without system delays.

7. **Security:**

   Code execution occurs in isolated sandbox environments, ensuring no unauthorized access or interference with server resources. The use of JWT-based authentication provides secure login and controlled data access.

3.3 Solution Approach

The CodeRoom platform is designed with a modular and scalable architecture that integrates modern web technologies to ensure seamless execution, security, and performance. The solution approach divides the system into four major components — Frontend, Backend, Database, and Execution Layer — all of which interact through structured APIs to deliver an efficient end-to-end workflow.

Frontend (React.js + TailwindCSS)

The frontend forms the user-facing layer of the system, built using React.js for a fast,

dynamic interface and TailwindCSS for responsive, minimalist styling.

- It includes two main dashboards - a Professor Dashboard and a Student Dashboard.
- The Professor Dashboard enables easy creation of assignments, management of test cases, and monitoring of student submissions and results.
- The Student Dashboard integrates a Monaco or CodeMirror-based online code editor, allowing students to write, compile, and execute code directly from the browser without any additional setup.

  This layer emphasizes user experience, offering smooth navigation and instant feedback through real-time updates from the backend.

Backend (Node.js + Express.js)

The backend acts as the central logic and communication hub of the platform. Developed using Node.js with the Express.js framework, it is responsible for managing users, assignments, and submission data.

- It exposes a set of RESTful APIs that handle CRUD operations (Create, Read, Update, Delete) for users, problems, and test cases.
- The backend securely integrates with the Judge0 API to forward code submissions for execution and fetch results asynchronously.
- It also handles authentication using JWT tokens, ensuring only verified users can access specific functionalities.

  Through asynchronous handling, the backend supports concurrent requests, maintaining smooth operation even when multiple users are compiling or submitting code simultaneously.

Database (MongoDB)

The platform uses MongoDB, a document-oriented NoSQL database, for efficient data storage and retrieval. It maintains multiple collections to organize system data effectively, including:

- Users – for storing student and professor credentials, roles, and access permissions.
- Problems – for maintaining coding questions, descriptions, and difficulty levels.
- TestCases – for holding input-output pairs used for evaluating student submissions.
- Submissions – for storing results, execution status, and performance metrics.

  MongoDB's flexible schema design and indexing enable high-speed data operations, while its aggregation pipelines power the analytics and reporting modules integrated into the professor dashboard.

Execution Layer (Judge0 API)

The Judge0 API serves as the backbone for code execution and evaluation. It provides a secure, containerized, and language-independent environment for running submitted programs.

- When a student submits their code, the backend sends the source code, input data, and selected language ID to Judge0 through a REST API request.
- Judge0 then executes the program inside a Docker-based sandbox, ensuring isolation from the main server.
- The API returns results such as output (stdout), error messages (stderr), execution time, and memory usage, which are then used to compute marks and determine verdicts like Accepted, Wrong Answer, or Runtime Error. This architecture ensures fairness, security, and performance while handling multiple concurrent submissions efficiently.

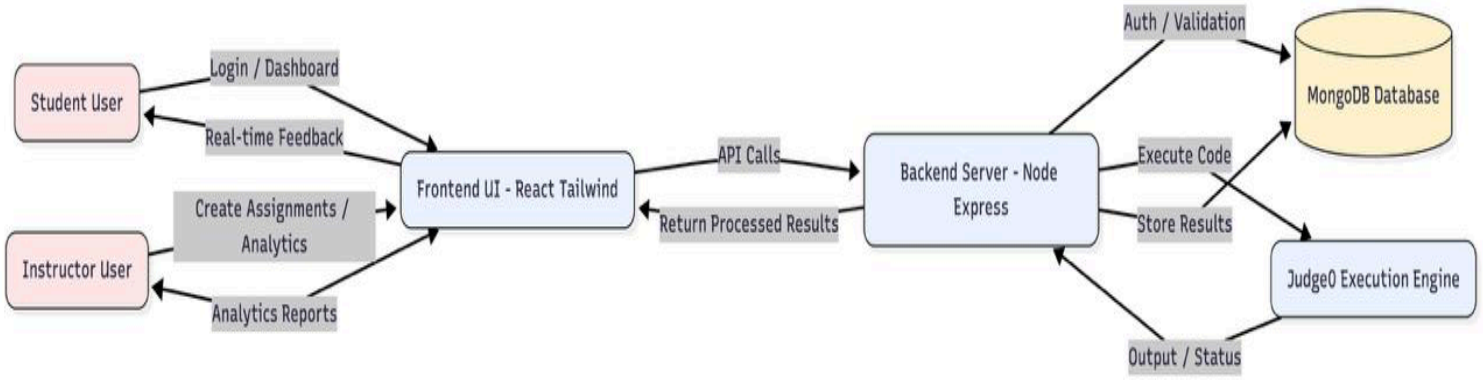


Figure 3.1-System Architecture of CodeRoom

Chapter 4

MODELING AND IMPLEMENTATION DETAILS

4.1 Design Diagrams

4.1.1 Control Flow Diagram

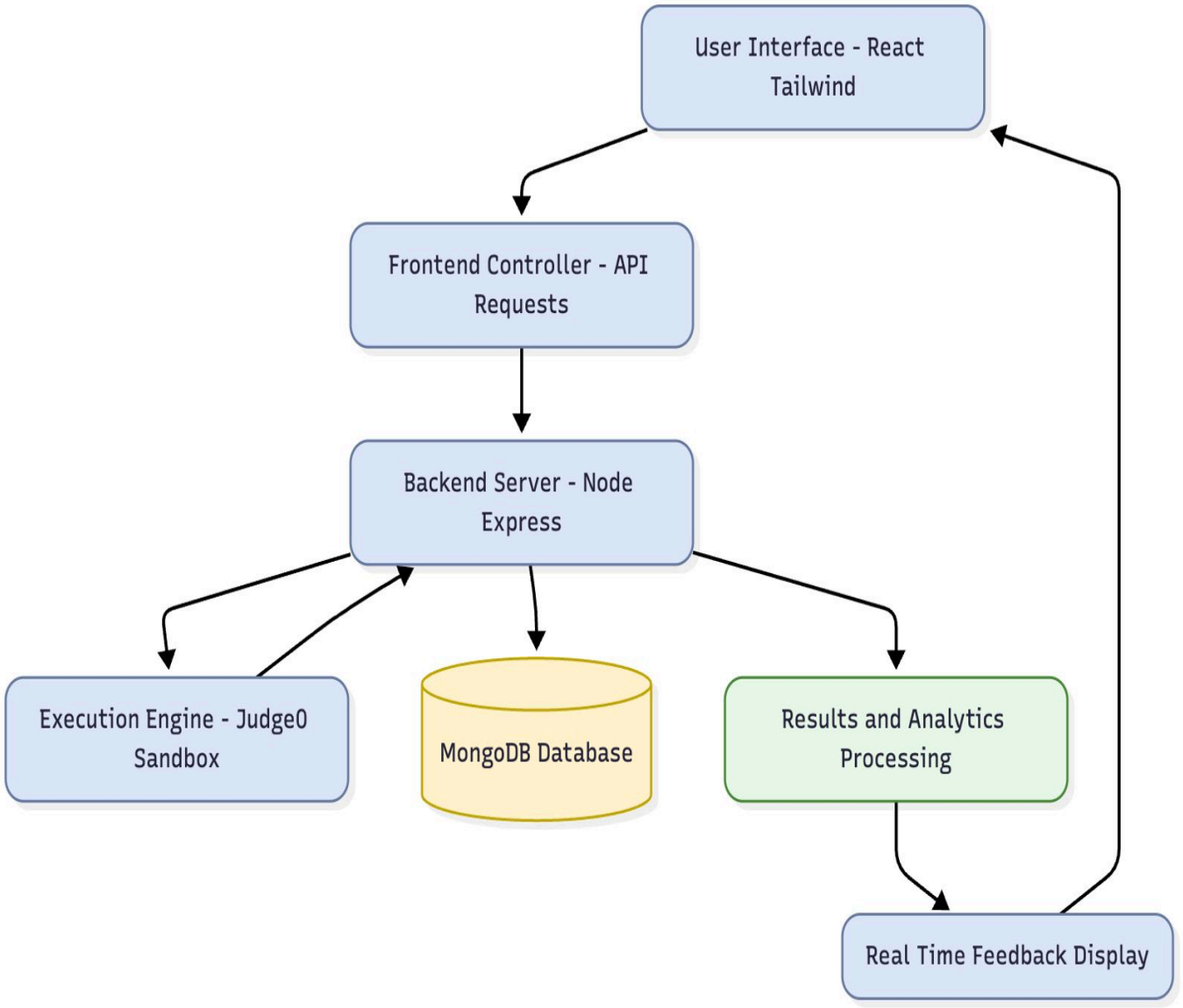


Figure 4.1-Control Flow Diagram of CodeRoom

Explanation of the Control Flow Diagram

The control flow diagram illustrates the complete operational sequence of the CodeRoom system, beginning from user interaction and extending through backend processing, code execution, data storage, and final feedback delivery. The flow starts with the User Interface (React and Tailwind), where both students and instructors interact with the platform. Users can log in, access their dashboards, and perform tasks such as submitting code, viewing assignments, or checking evaluation results. All user actions are forwarded to the Frontend Controller, which acts as a communication bridge between the interface and the backend. It packages user requests into API calls and ensures they are transmitted securely to the server.

The Backend Server (Node.js and Express) forms the core processing unit of the system. It handles all API endpoints, validates user input, routes submissions, and coordinates communication between different system components. When a student submits code for evaluation, the backend forwards this request to the Execution Engine (Judge0 Sandbox). Judge0 executes the code inside a secure, isolated container, ensuring that untrusted student programs do not pose any threat to the application environment. The execution engine returns outputs, runtime errors, or compilation messages back to the backend in a structured format.

Upon receiving these results, the backend performs two critical operations: storing the submission data and generating real-time insight. The output is first saved in the MongoDB Database, allowing the system to maintain historical records, generate reports, and support analytics. Simultaneously, the data is forwarded to the Results and Analytics Processing module, which transforms raw execution results into meaningful insights such as performance indicators, error patterns, and progress metrics. These processed results are then delivered to the Real-Time Feedback Display, allowing students to instantly view their outputs, errors, and grading information.

Finally, the feedback is sent back to the User Interface, completing the loop. This creates a seamless, interactive system where users receive rapid, consistent, and accurate responses. Overall, the control flow diagram demonstrates the coordinated functioning of multiple layers working together to deliver automated code evaluation, ensuring a secure, scalable, and learner-centric experience.

4.1.2 Workflow Diagram

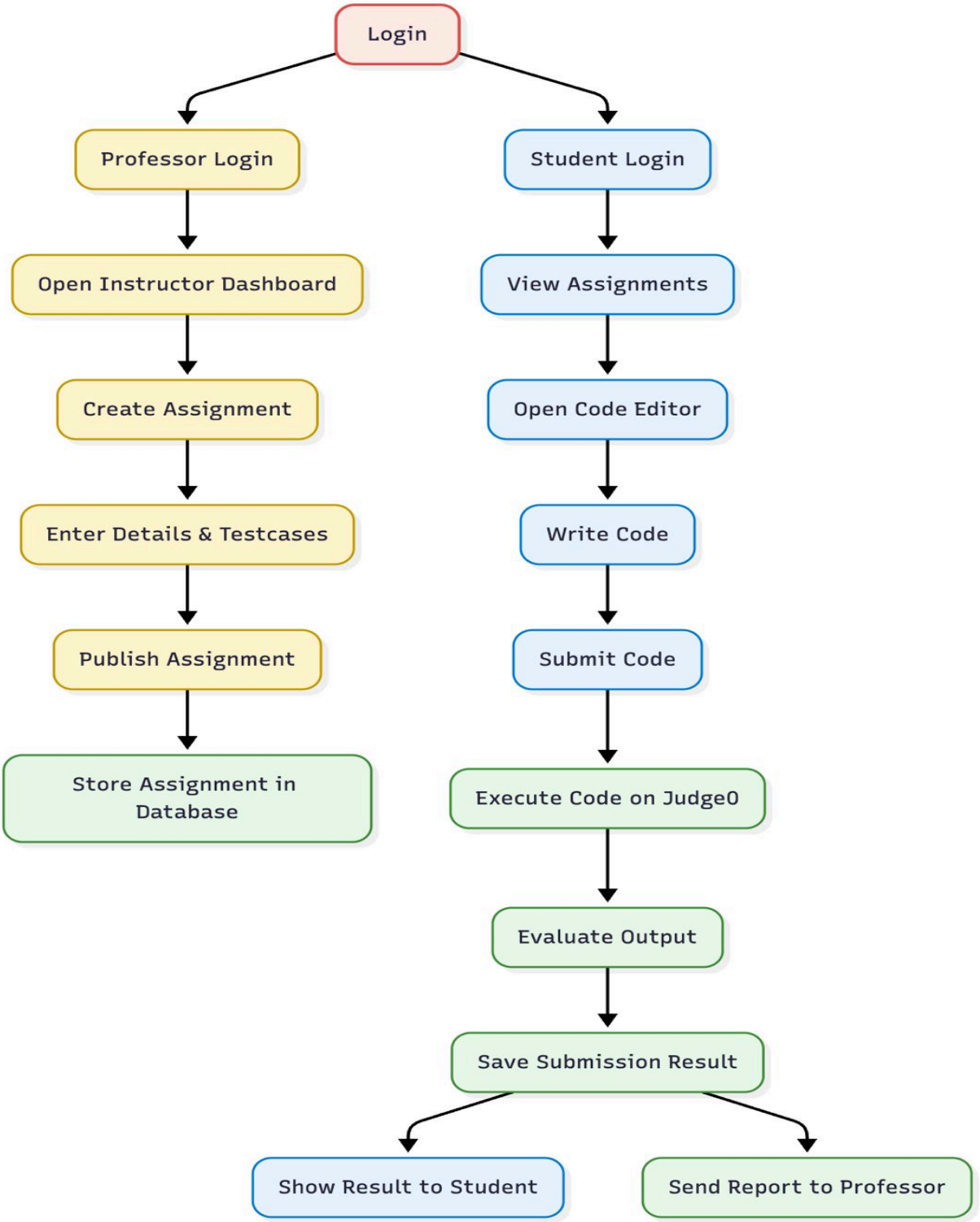


Figure 4.2-Work Flow Diagram of CodeRoom

Explanation of the Workflow Diagram

The workflow diagram outlines the end-to-end functioning of the CodeRoom platform, beginning at the Login stage and then dividing into two parallel workflows for Professors and Students. After logging in, professors are directed to the Instructor Dashboard, where they can create new programming assignments by entering problem details and configuring both sample and hidden test cases. Once an assignment is published, it is stored in the database and becomes immediately accessible to all students, ensuring a smooth and organized academic workflow.

Students, on the other hand, access the Student Dashboard after logging in, where they can view the list of available assignments. Upon selecting an assignment, they are taken to the integrated code editor to write their solution. When the student submits their code, the system forwards it to the backend, which sends the program for secure execution to the Judge0 sandbox engine. Judge0 compiles and runs the code in an isolated environment, returning either successful outputs or error information depending on the submission.

After receiving the execution results, the backend evaluates them against the predefined test cases and records the submission details in the database. The processed result is then sent back to the student interface, providing immediate feedback to help learners identify mistakes and improve their solutions. Additionally, the system generates a summary report for the professor, enabling them to monitor individual and class-wide performance.

Overall, the workflow diagram demonstrates how CodeRoom manages two coordinated user journeys—one centered on assignment creation and academic oversight, and the other focused on code writing, automated evaluation, and iterative learning. It highlights the seamless interaction between the frontend, backend, database, and execution engine, showing CodeRoom as a cohesive and efficient coding assessment platform.

4.1.3 Use Case Diagram

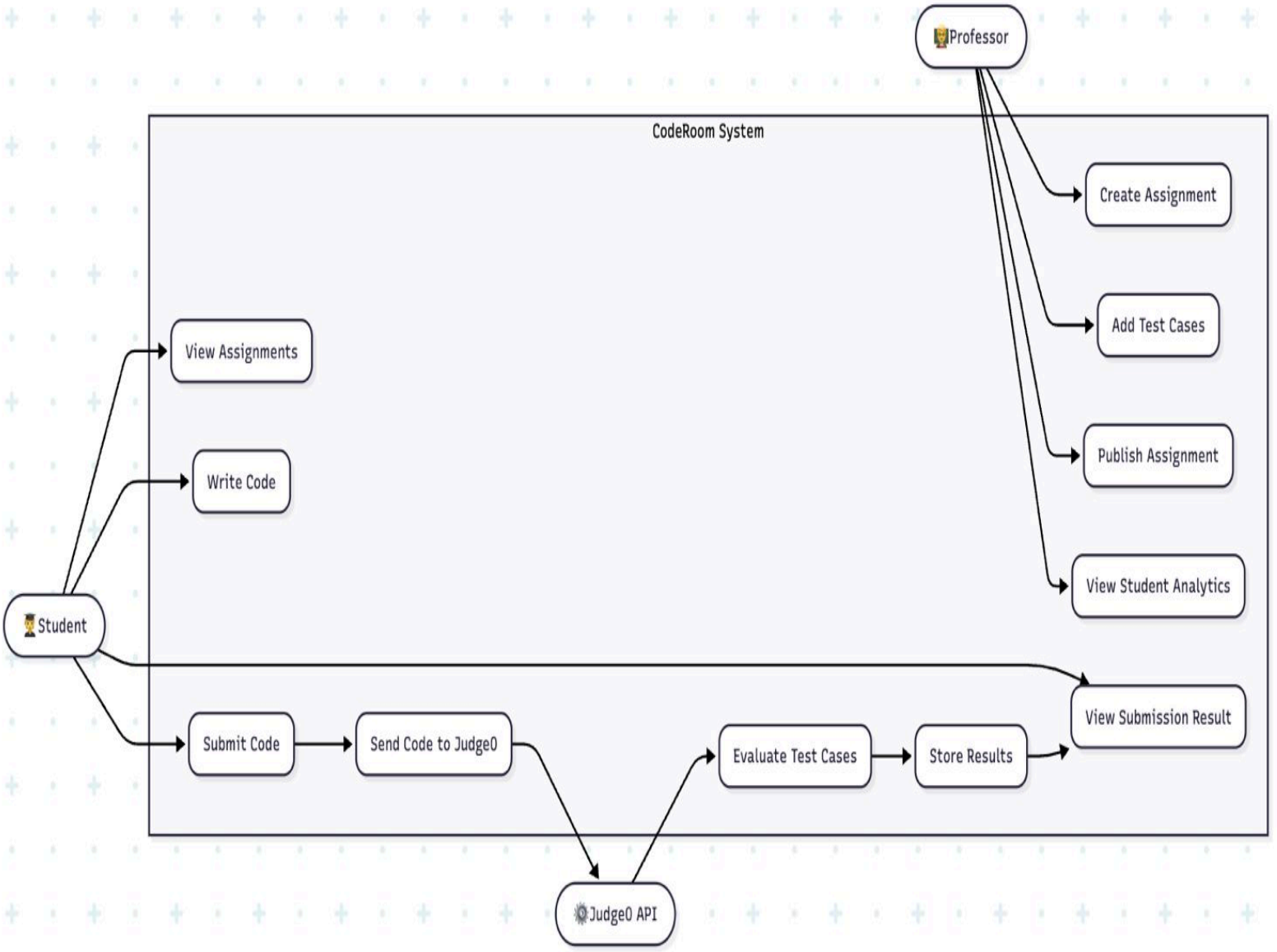


Figure 4.3-Use Case Diagram of CodeRoom

Explanation of the Use Case Diagram

The workflow diagram provides a clear representation of how the CodeRoom system operates by depicting the interactions between students, instructors, and the internal system components. The diagram shows both user roles acting on the platform and highlights how their actions move through different stages of the system. On the student side, learners can view assignments, write code, and submit their solutions for evaluation. These actions

represent the core student workflow and are visually separated to emphasize the student's learning and coding process.

When a student submits code, the flow transitions into the system's responsibility. The submission is first accepted by CodeRoom and then forwarded to the Judge0 engine for secure execution. Judge0 compiles and runs the program in an isolated sandbox environment to ensure consistency and safety. After execution, the results flow back into the CodeRoom system where the test cases are evaluated, and the outcome is stored in the database. The final processed results are then displayed to the student under "View Submission Result," completing their workflow loop.

On the instructor side, the workflow highlights the academic control features available to professors. Instructors can create new assignments, define and configure test cases, and publish assignments for students to access. They can also view detailed student analytics, allowing them to track performance and identify students who may need additional guidance. These instructor actions flow directly into the CodeRoom system, where the associated data is stored, managed, and made available to the student interface.

Overall, the workflow diagram illustrates how CodeRoom manages two parallel but interconnected user journeys. Students interact primarily with assignments and code execution, while instructors manage content creation and performance evaluation. Both workflows rely on the CodeRoom system's backend logic and the Judge0 engine to handle secure code execution, result processing, and data storage. This integration ensures a smooth, structured, and efficient environment for both learning and teaching within the platform.

4.1.4 Class Diagram

Frontend UML diagram

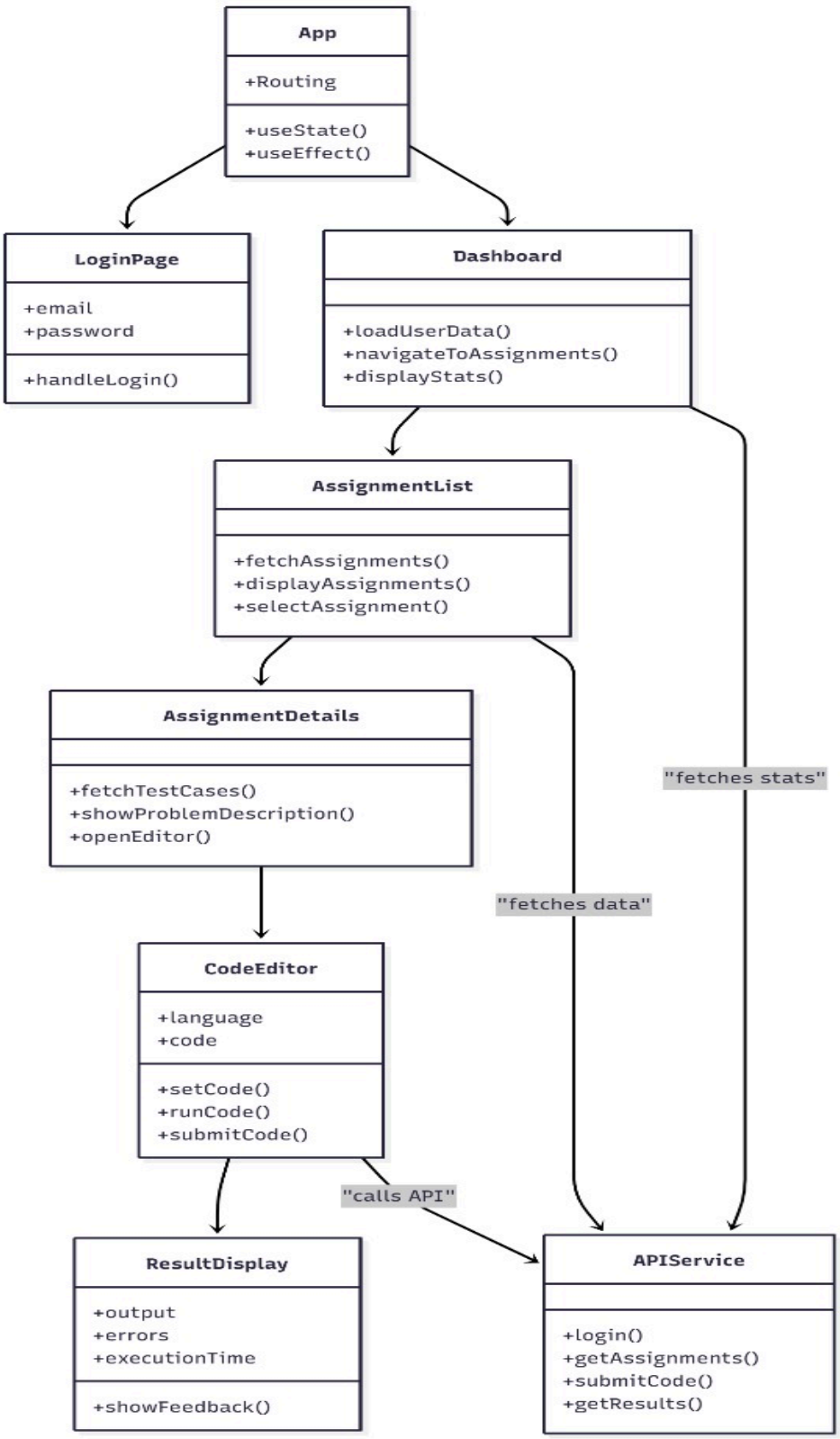


Figure 4.4-Frontend UML Diagram of CodeRoom

Explanation of the Frontend UML Diagram

The UML diagram of the frontend architecture illustrates how the main React components in CodeRoom work together to create a smooth user experience. The App component serves as the entry point, handling routing, initializing global states, and deciding whether the user is directed to the LoginPage or the Dashboard. The LoginPage manages user authentication by sending credentials to the backend through the APIService.

Once authenticated, the user is taken to the Dashboard, which functions as the central hub for displaying assignments, performance insights, and navigation options. It retrieves updated data through the APIService. From here, users can access the AssignmentList component to view all available tasks, and then open the AssignmentDetails component to see problem descriptions and test case summaries.

The AssignmentDetails component leads to the CodeEditor, where students can write and submit code. The CodeEditor manages code input, language selection, and communicates with the backend for execution. After submission, results are displayed through the ResultDisplay component, which shows outputs, errors, and execution feedback.

Throughout the entire workflow, the APIService ensures smooth communication between the frontend and backend. The diagram highlights a clean, modular structure where each component has a clear responsibility, supporting maintainability and easy future expansion.

Backend UML diagram

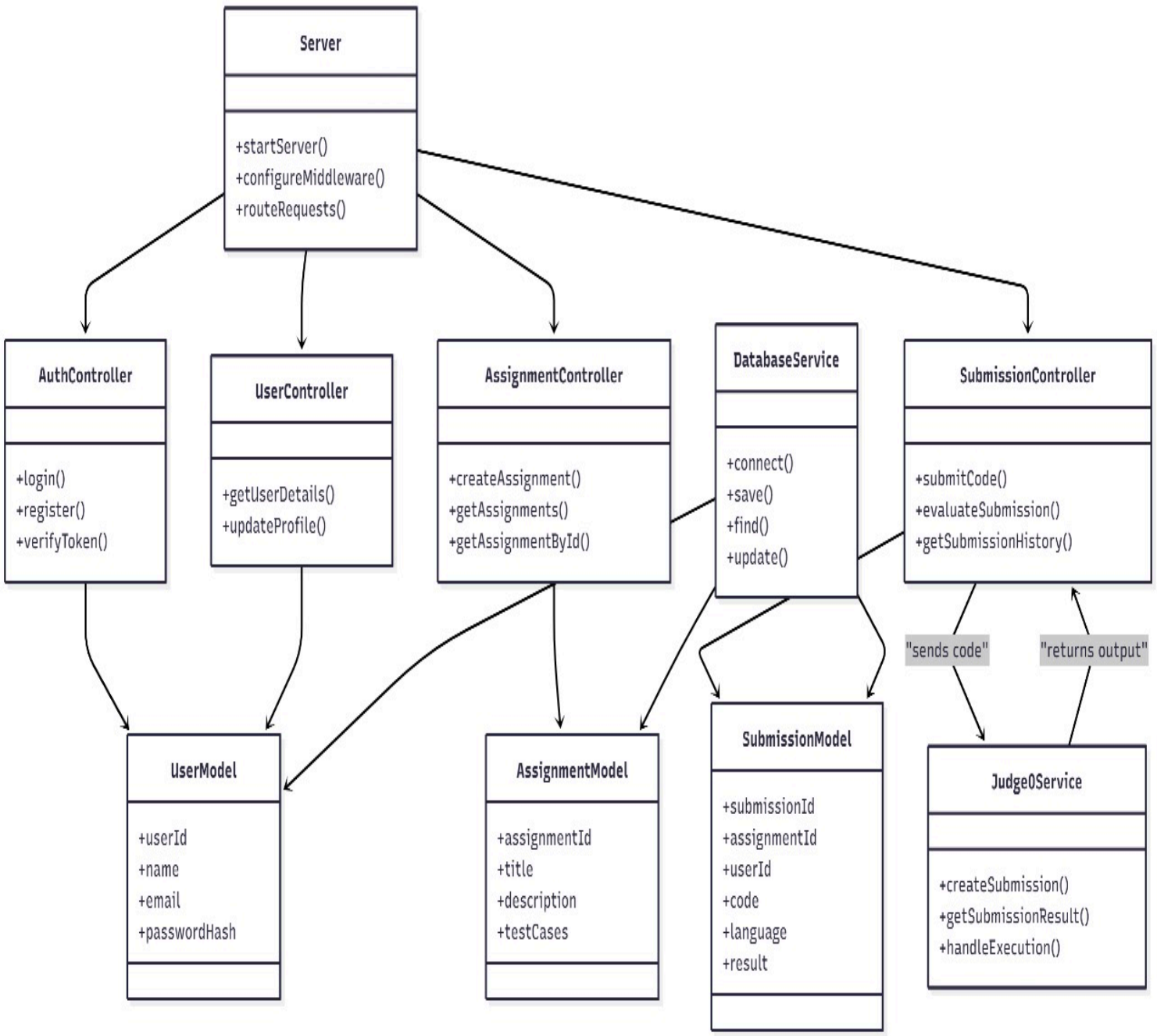


Figure 4.5-Backend UML Diagram of CodeRoom

Explanation of the Backend UML Diagram

The backend UML diagram provides a structured overview of CodeRoom's server-side architecture by illustrating how controllers, services, and database models work together to support secure code execution and data management. At the core is the Server component, built using Node.js and Express, which initializes the application, configures middleware, and routes all incoming API requests to the appropriate controller. This ensures organized request handling and smooth backend operation.

The controller layer is modular and divided based on functionality. The AuthController manages login, registration, and token verification, ensuring secure access. The UserController handles user profile operations, while the AssignmentController allows instructors to create and retrieve assignments. The SubmissionController oversees the submission workflow, forwarding student code for execution and storing the results.

Code execution is handled by the Judge0Service, which communicates with the external Judge0 engine to execute code securely in a sandboxed environment. Once execution results are retrieved, they are returned to the submission controller for processing. The backend also includes a DatabaseService responsible for managing MongoDB connections and performing CRUD operations on stored data.

Data is organized into three models- User Model, Assignment Model, and Submission Mode each defining a structured schema for users, assignments, and code submissions. These models ensure that all data remains organized, consistent, and easy to query.

Overall, the backend UML diagram highlights a modular and scalable structure where each component has a clear responsibility. This separation of controllers, services, and models enhances maintainability, supports secure code execution, and provides a reliable foundation for CodeRoom's real-time evaluation features.

4.1.4 Sequence Diagram

Student Code Sequence Diagram

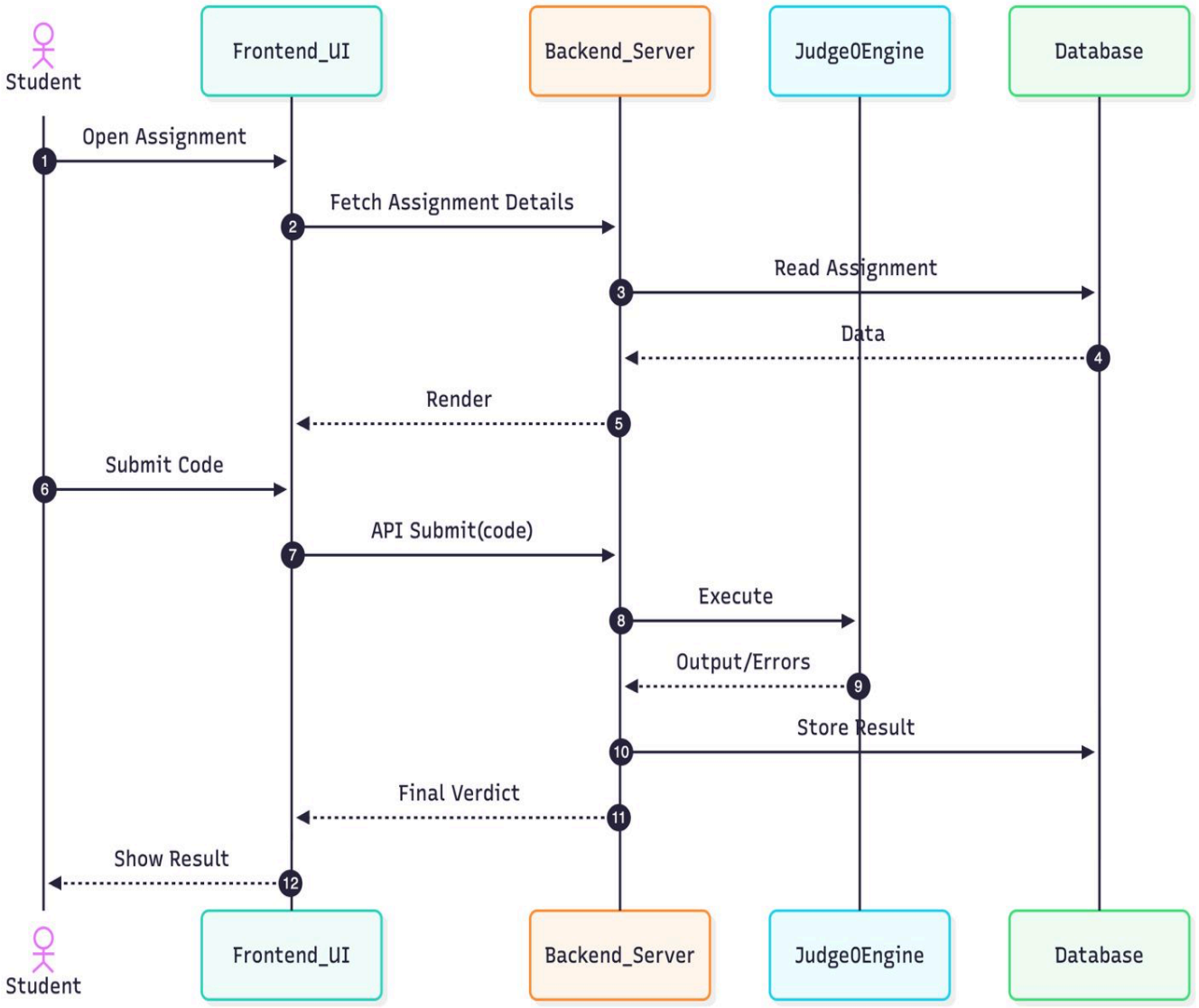


Figure 4.6-Student Code Sequence Diagram of CodeRoom

Explanation of the Student Sequence Diagram

The sequence diagram illustrates the step-by-step interaction between the Student, Frontend UI, Backend Server, Judge0 Engine, and the Database during a typical assignment execution workflow in CodeRoom. The flow begins when the student selects an assignment from the

interface. This action triggers a request from the Frontend UI to the Backend Server to fetch assignment details. The backend then queries the Database, retrieves the required data, and sends it back to the frontend to be rendered for the student.

Once the student reviews the problem and writes their code, they submit the solution through the Frontend UI. The frontend forwards this submission to the Backend Server via an API call. Upon receiving the code, the backend routes it to the Judge0 Engine for secure execution in an isolated sandbox environment. Judge0 compiles and runs the program, returning either outputs or error information back to the backend.

The Backend Server then evaluates the returned output, saves the execution results to the Database for record-keeping, and prepares a final verdict for the student. This verdict—whether successful execution or an error—is sent back to the Frontend UI. Finally, the interface displays the result to the student, completing the cycle.

Overall, the sequence diagram highlights the synchronous flow of data across multiple system layers and clearly demonstrates how CodeRoom manages assignment retrieval, code submission, execution, and result feedback in a coordinated, efficient manner.

Professor Code Sequence Diagram

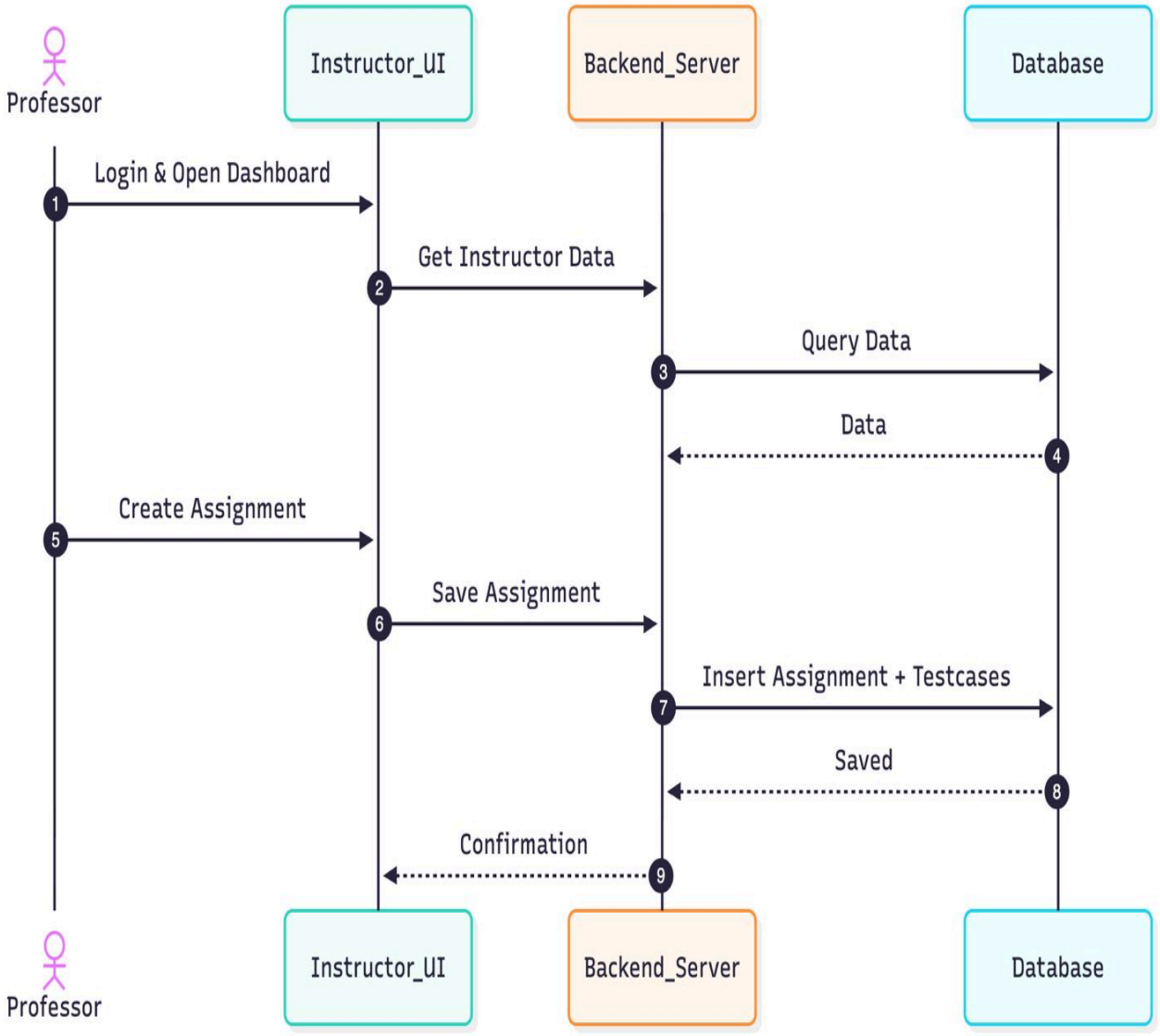


Figure 4.7-Professor Code Sequence Diagram of CodeRoom

Explanation of the Professor Sequence Diagram

The sequence diagram illustrates the complete flow of how a professor creates and saves a new programming assignment in the CodeRoom platform. The interaction begins when the professor logs in and opens the instructor dashboard. This action triggers the Instructor UI to request the professor's data from the Backend Server. The backend then queries the Database, retrieves the necessary instructor information, and sends it back to the UI for display.

Once the dashboard is loaded, the professor initiates the creation of a new assignment. This request is sent from the Instructor UI to the Backend Server, which processes the assignment

details and prepares them for database storage. The backend then sends a command to the Database to insert the new assignment along with its associated test cases. The Database stores the information and returns a confirmation response to the backend.

After receiving the successful save confirmation, the Backend Server sends a final acknowledgment back to the Instructor UI. This confirmation informs the professor that the assignment has been successfully created and stored, completing the workflow.

Overall, this sequence diagram demonstrates how CodeRoom ensures smooth coordination between the instructor interface, backend logic, and database operations. It highlights the platform's ability to securely store assignment data while maintaining an organized, interactive experience for instructors.

4.2 Risk Analysis and Mitigation

The CodeRoom platform involves multiple interacting components—frontend, backend, database, and an external code execution service. Due to this complexity, several risks may arise that could impact security, performance, reliability, or user experience. This section outlines the major risks identified during development and the strategies implemented to mitigate them effectively.

1. Security Risks

Risk: Unauthorized access poses a significant risk to the CodeRoom platform because it can expose sensitive information such as user accounts, assignments, and submission data. If authentication or API endpoints are not properly secured, attackers may attempt to bypass login mechanisms, exploit weak tokens, or manipulate unprotected routes to gain elevated access. Such breaches could allow unauthorized users to view or alter test cases, modify assignment data, or access student submissions, ultimately compromising academic integrity. This makes strong authentication, encrypted communication, and strict access control essential for maintaining system security.

Mitigation:

- JWT-based secure authentication
- Password hashing using bcrypt
- Role-based access control for students and instructors
- HTTPS enforcement for API communication

2. Code Execution Threats

Risk: User-submitted code can be unpredictable and may include dangerous instructions such as infinite loops, memory-heavy operations, or attempts to access restricted files. If not properly sandboxed, such code could overload system resources or pose security threats. Because CodeRoom executes many submissions automatically, any harmful code could impact system performance or stability. Using a secure sandbox environment like Judge0 helps ensure that all programs run in isolation, protecting the core system from potential damage.

Mitigation:

- Judge0 sandbox ensures isolated execution
- Strict memory, time, and CPU limits
- No direct filesystem or network access for user code

3. Data Loss or Corruption

Risk: Another concern is the possibility of losing important academic data due to database failures, incorrect writes, or structural inconsistency. Assignments, test cases, and submission histories are crucial for evaluation and audits. If any of this data becomes corrupted, incomplete, or accidentally deleted, it can affect both students and instructors. Reliable backups, schema validation, and safe write operations are essential to maintain data integrity.

Mitigation:

- Regular database backups
- Validation layers in Mongoose models

- Redundant storage and fail-safe write operations

4. Performance Bottlenecks

Risk: During peak usage—especially near assignment deadlines—the number of submissions can increase rapidly. This surge may slow down the backend server, delay code execution, or cause timeouts. Heavy traffic can also affect database query speeds and reduce overall system responsiveness. To maintain a good user experience, the platform must include mechanisms like caching, workload distribution, and optimized queries to handle high demand efficiently.

Mitigation:

- Load balancing for execution requests
- Caching frequently accessed data
- Optimized MongoDB indexing

5. API Reliability Issues

Risk: Because CodeRoom depends on Judge0 for code execution, any downtime, slow response, or API rate limit issues can interrupt the submission workflow. If Judge0 becomes temporarily unavailable, students may be unable to execute or evaluate their code, leading to frustration and delays. Implementing retry logic, proper error handling, and optional fallback mechanisms helps reduce dependency issues and ensures the system remains stable even when external services slow down.

Mitigation:

- Retry and fallback mechanisms
- Graceful error handling and user notifications
- Optional local Judge0 deployment during outages

6. Scalability Risks

Risk:

As more institutions and students begin using CodeRoom, the number of assignments, submissions, and execution requests may increase rapidly. Without a scalable architecture, the system could struggle under high traffic.

Mitigation:

- Modular microservice-friendly architecture
- Horizontal scaling options for backend & database
- Efficient API design minimizing payload load

Chapter – 5

TESTING

5.1 Testing Plan

Testing plays a crucial role in ensuring that the CodeRoom platform performs reliably across different devices, user roles, and operating conditions. The main objective of the testing plan is to verify that each module works correctly on its own and functions smoothly when integrated into the complete system. To achieve this, testing was carried out in two structured phases: individual testing during development and large-scale testing on the campus server.

Phase A - Individual / Self Testing

This phase was carried out continuously throughout the development cycle. Each time a new feature was added or an existing module was updated, it was tested independently to confirm that it behaved as expected. The purpose of this phase was to detect issues early, fix them quickly, and maintain a stable development workflow.

During self-testing, the following major components were validated:

- **User Interface Behavior:** Ensured that pages loaded correctly, buttons functioned properly, and navigation between screens was smooth and intuitive.
- **Backend API Functionality:** Verified that all API routes responded with correct data, returned accurate status codes, and handled unexpected inputs safely.
- **Database Schema and Operations:** Checked whether MongoDB correctly stored, updated, and retrieved user, assignment, and submission data without inconsistencies.
- **Code Execution and Evaluation Engine:** Confirmed that student programs were compiled and executed securely using Judge0, and that the results matched expected behavior.
- **Error Handling for Invalid Submissions:** Tested how the system responded to syntax errors, runtime failures, infinite loops, and missing fields to ensure that students receive clear and helpful feedback.

Overall, Phase A ensured that every module was internally stable before being combined with others, lowering the chances of major issues during large-scale deployment.

Phase B - On-Campus Server Deployment (30 Students)

The on-campus deployment conducted in the JIIT Sector-128 Data Structures Lab provided a real academic environment to evaluate how CodeRoom performs under actual classroom conditions. Unlike controlled self-testing, this phase tested the platform with 30 students interacting simultaneously—logging in, navigating the dashboard, accessing assignments, and submitting code at the same time. This allowed us to assess not only the technical stability of the platform but also how intuitive and responsive it feels to real users. Through this test, we monitored system behavior under load, measured actual execution times, identified real usage bottlenecks, and collected direct feedback from students and instructors. The objective was to ensure that CodeRoom is not only technically sound but also practically usable in a live teaching environment.

Objectives of Real-World Testing

- **Evaluate concurrent system performance:**
  To determine whether the backend, database, and execution engine can collectively handle multiple logins, assignment fetches, and code submissions without experiencing noticeable delays or crashes.
- **Test complete academic workflow:**
  To verify that the instructor can create assignments and that students can access, attempt, and submit them end-to-end without encountering functional obstacles or inconsistencies.
- **Validate real-time code execution under load:**
  To ensure that Judge0 continues to execute code accurately and return outputs promptly even when several users submit their programs simultaneously, simulating real classroom load.
- **Monitor backend stability and response time:**
  To observe how the server manages parallel requests, database reads/writes, and execution tasks, and to confirm that response times remain within an acceptable range for students.

- **Identify front-end usability improvements:**

  To check how quickly the UI responds during heavy usage, whether any screens lag, and how easily students navigate between assignments, code editor, and results.
- **Verify integration of all system modules:**

  To ensure smooth communication between frontend components, backend controllers, MongoDB models, and the Judge0 execution service under simultaneous user activity.
- **Evaluate handling of diverse code behaviors:**

  To test how the system manages real student submissions—including correct code, syntax errors, infinite loops, and memory-heavy programs—to verify that error reporting remains consistent.
- **Collect real user feedback:**

  To gather genuine reactions from students regarding usability, clarity of the interface, and satisfaction with the quick feedback cycle, as these insights help refine the platform for wider deployment.

5.2 Component Decomposition and Type of Testing Required

To ensure thorough and systematic testing of CodeRoom, the platform was decomposed into smaller functional components. Each component was examined individually and then tested in combination to verify its role in the complete system. This approach helped isolate potential issues early and ensured that integration did not introduce new errors. Breaking the system into frontend, backend, database, and execution service modules allowed us to understand their responsibilities clearly and determine the exact type of testing required for each.

5.2.1 Component Decomposition

- **Frontend Components:**

  Includes the Login Page, Dashboard, Assignment List, Assignment Details, Code Editor, and Result Display. These are responsible for user interactions, UI behaviour, and immediate responses to student/instructor actions.
- **Backend Server Components:**

  Consist of controllers (Auth, User, Assignment, Submission), services (Judge0 service, database service), and middleware. These handle authentication, assignment creation,

submission processing, execution routing, and analytics generation.

- **Database Layer:**

  Composed of MongoDB collections such as UserModel, AssignmentModel, and SubmissionModel. They store user profiles, assignment metadata, test cases, and execution results.

- **Execution Engine:**

  Judge0 API, which compiles and executes code in a secure containerized environment. It plays a crucial role in ensuring safe and consistent evaluation of student submissions.

5.2.2 Type of Testing Required

- **Unit Testing:**

  Applied to individual functions and modules such as login validation, assignment creation logic, database queries, and code submission handlers. This ensures each component works correctly by itself before integration.

- **Integration Testing:**

  Focuses on evaluating how combined modules behave together—for example, testing the full flow from assignment creation to code submission to result display. It ensures that the handoff between frontend, backend, database, and execution engine is smooth.

- **Functional Testing:**

  Ensures that all system features align with the SRS. This includes assignment fetching, code execution, result display, analytics, and instructor tools.

- **Performance Testing:**

  Conducted to evaluate how well the system handles multiple users and heavy activity. It measures execution time, UI responsiveness, and server stability.

- **Stress & Load Testing:**

  Performed by submitting long-running code, memory-intensive programs, and numerous simultaneous submissions to test how the system handles extreme cases.

- **User Interface Testing:**

  Ensures that the layout is clean, pages are responsive, and navigation is intuitive across devices. It checks button actions, error notification behavior, and input validation.

- **Error Handling Tests:**

  Verifies that the system gracefully handles syntax errors, infinite loops (TLE), memory

overflow (MLE), runtime crashes, missing input fields, and invalid API requests.

5.3 List of Test Cases

Test Cases for Login & Authentication (Component 1)

Table 5.1-Test Cases for Login & Authentication

Test Case ID	Input	Expected Output	Status
TC-L01	Valid email + password	Login successful, redirect to dashboard	Pass
TC-L02	Valid email + wrong password	Error: “Invalid Credentials”	Pass
TC-L03	Empty fields	Error: “Fields cannot be empty”	Pass

Test Cases for Dashboard & Navigation (Component 2)

Table 5.2-Test Cases for Dashboard and Navigation

Test Case ID	Input	Expected Output	Status
TC-D01	Logged-in user opens dashboard	Dashboard loads successfully	Pass
TC-D02	Session expired	Redirect to login	Pass
TC-D03	Navigate Assignment → Editor	Correct page renders	Pass

Test Cases for Assignment Management (Component 3)

Table 5.3-Test Cases for Assignment Management

ID	Input	Expected Output	Status
TC-A01	Valid title, description, test cases	Assignment created	Pass
TC-A02	Missing title	Validation error	Pass
TC-A03	Duplicate assignment	Error message	Pass

Test Cases for Code Editor (Component 4)

Table 5.4-Test Cases for Code Editor

ID	Input	Expected Output	Status
TC-E01	Enter valid code	Code accepted	Pass
TC-E02	Empty code	Show “Code cannot be empty”	Pass
TC-E03	Random invalid characters	Show error	Pass

Test Cases for Code Submission & Evaluation (Component 5)

Table 5.5-Test Cases for Code Submission and Evaluation

ID	Input	Expected Output	Status
TC-S01	Correct code	AC (Accepted)	Pass
TC-S02	Syntax error	CE	Pass
TC-S03	while(1){}	TLE	Pass
TC-S04	Memory-heavy program	MLE	Pass

Test Cases for Judge0 Integration (Component 6)

Table 5.6-Test Cases for Judge0 Integration

ID	Input	Expected Output	Status
TC-J01	Valid code payload	Output returned	Pass
TC-J02	Invalid language ID	Error message	Pass

Test Cases for Backend API Controllers (Component 7)

Table 5.7-Test Cases for Backend API Controllers

ID	Input	Expected Output	Status
TC-B01	GET /assignments	Assignment list	Pass
TC-B02	POST /submit	Result JSON	Pass

Test Cases for Database Layer (Component 8)

Table 5.8-Test Cases for Database Layer

ID	Input	Expected Output	Status
TC-DB01	Valid submission object	Stored in MongoDB	Pass
TC-DB02	Missing required field	Validation error	Pass

5.4 Error and Exception Handling

Error and exception handling was a crucial part of development because CodeRoom frequently executes user-generated code, which can cause unpredictable behavior. Every stage—from assignment creation to code execution—was tested to ensure the system could detect and manage failures gracefully, without crashing or corrupting data.

5.4.1 Backend Errors

- **Compilation Errors (CE):**
  These occur when students submit code with syntax errors. Judge0 returns compiler output, which CodeRoom formats and displays cleanly so students understand the issue.
- **Runtime Errors (RE):**
  These include division by zero, invalid memory access, or segmentation faults. Such errors are isolated during execution and returned without affecting server stability.
- **Time Limit Exceeded (TLE):**
  Infinite loops or highly inefficient algorithms trigger the timeout. The system enforces

strict limits to prevent server overload.

- **Memory Limit Exceeded (MLE):**

  Excessive memory usage or deep recursion leads to memory caps being exceeded. The sandbox stops execution immediately and returns MLE to the user.

- **Database Exceptions:**

  Schema mismatches, missing fields, and invalid data types were handled using Mongoose validation. All incoming data is sanitized before saving.

5.4.2 Frontend Errors

- **Form Validation Errors:**

  Occurred when mandatory fields were left blank or invalid input was provided. Real-time validation ensures users correct errors before submission.

- **State Management Errors:**

  Rapid clicks or user actions sometimes led to inconsistent component states. Debouncing user actions and improving state transitions resolved this.

- **API Handling Errors:**

  Network delays or server downtime caused temporary failures. These were handled by adding retry logic, toast notifications, and fallback UI states.

5.4.3 Deployment & Server Errors

- **Docker Build Issues:**

  Missing dependencies or incorrect build configurations caused container failures. Multi-stage Docker builds resolved these issues.

- **LAN Latency Problems:**

  During campus testing, a few students experienced slow logins due to network congestion. Reducing unnecessary API calls and enabling caching solved the issue.

5.5 Limitations of the Solution

While CodeRoom performs reliably under small to medium academic loads, there are certain limitations in the current version:

- **Limited Scaling Capacity:**
  The current architecture supports roughly 30–50 users simultaneously. For larger deployments, load balancing and distributed execution would be required.

- **Dependence on Judge0:**
  The external execution engine is essential for compiling and running programs. Downtime or delays in Judge0 directly impact CodeRoom's functionality.
- **Basic Interface Customization:**
  The current UI does not include advanced customization features like dark mode or theme control, which some users requested.
- **Basic Analytics:**
  Instructor analytics are functional but limited. More advanced features like plagiarism detection, progress visualization, and AI-based hints are not yet included.
- **Network Dependency:**
  Performance during campus testing was partially influenced by LAN conditions. Heavy network traffic can slow down the experience for some users.

Testing Verification Certificate

This is to certify that the application CodeRoom was successfully tested on-campus at Jaypee Institute of Information Technology, Sector 128, during DS Week – Lab 5, with participation from ~30 students.

The testing validated system functionality, performance, stability, and user experience. The test involved live usage by students and instructors, including assignment creation, code submission, and automatic evaluation.

This document serves as an official acknowledgment of the testing process conducted within the institute premises.

_____

Responsible Authoritie(s)

JIIT Sector 128

Chapter-6

FINDINGS, CONCLUSIONS AND FUTURE WORK

6.1 Findings

After completing the development and thorough testing of the CodeRoom platform—both individually and in a real classroom environment—several important findings were identified that highlight the effectiveness and reliability of the system.

- **Improved Evaluation Efficiency**

  The platform significantly reduced the workload associated with manual checking. Instructors reported that the automated judging system allowed them to evaluate an entire batch of submissions within minutes. The system's instant verdicts, structured evaluation workflow, and automatic result storage eliminated repetitive tasks and made the overall assessment process far more efficient.

- **Instant Feedback Accelerates Learning**

  Students greatly benefited from the immediate feedback provided after every submission. Compilation errors, runtime issues, logical mistakes, TLE, and MLE responses were displayed instantly, allowing students to debug their programs independently. This real-time feedback loop helped students learn faster and understand their mistakes without waiting for instructor intervention.

- **High System Stability During Concurrent Usage**

  During the on-campus testing session with 30 students working simultaneously, the system demonstrated strong stability. There were no crashes or downtimes, and every submission was processed and evaluated successfully. The Judge0 sandbox handled multiple parallel execution requests smoothly, proving that the architecture can support concurrent student activity in real classroom scenarios.

- **Accurate and Consistent Evaluation**

  Because all code was compiled and executed inside a controlled, identical sandbox environment, evaluations remained consistent for every student. No user relied on local machine configurations or compiler versions. This ensured fairness, eliminated system-related variations, and removed any potential bias from the evaluation process.

- **Strong Reliability of Backend–Execution Integration**

  The communication pipeline between the Express.js backend, Judge0 execution engine, and MongoDB database functioned reliably throughout testing. Even under load, the system handled code execution requests, stored submissions, and returned processed results without failure. Error categories such as CE, RE, TLE, and MLE were captured and displayed exactly as intended, confirming the reliability of the execution workflow.

- **Positive User Feedback**

  Students appreciated the clean interface, intuitive navigation, and smooth code editor experience. They found the feedback system especially helpful in improving their coding accuracy.

  Instructors highlighted the usefulness of the analytics dashboard, organized submission records, and the ease of creating and managing assignments. The platform gave them greater control and visibility over student progress in a structured and efficient manner.

6.2 Comparison with Traditional Evaluation

Table 6.1-CodeRoom VS Manual Evaluation

Parameter	Manual Evaluation	CodeRoom Automated Evaluation
Time Taken	Very High	Very Low
Consistency	Depends on Faculty	Fully Consistent
Feedback	Delayed	Instant
Scalability	Poor	High
Error Detection	Manual	Automated
Reporting	Manual Effort	Auto Generated

Key Observations

- CodeRoom drastically reduces faculty workload.
- Students learn faster due to the instant turnaround time.
- Uniform evaluation removes human bias.
- Systems like Google Classroom or Moodle lack automated execution—CodeRoom fills this gap.

6.3 Conclusion

• CodeRoom Achieves Its Core Objective

CodeRoom successfully delivers a secure, scalable, and automated coding evaluation platform tailored for academic usage. By integrating modern technologies and ensuring smooth interaction between all components, the system meets the needs of both students and instructors.

• Modern Technology Integration

The platform's architecture—combining React.js, Node.js/Express, MongoDB, and the Judge0 execution engine—creates a robust and efficient ecosystem. This integration ensures reliable performance, clean UI rendering, and fast, sandboxed code execution for multiple programming languages.

• Automated and Consistent Evaluation

Through sandbox-based execution, CodeRoom eliminates inconsistencies caused by local system variations. Every student program is executed in the same controlled environment, resulting in fair, unbiased, and accurate evaluations with instant feedback.

• Enhanced Learning Through Instant Feedback

Students benefit greatly from immediate responses to their code submissions. Clear CE, RE, TLE, and MLE messages guide learners toward debugging on their own, promoting independent learning and faster improvement in programming skills.

• Improved Instructor Productivity

Instructors can create assignments, add test cases, and monitor submissions with ease. Automated grading and organized analytics significantly reduce manual effort, transforming lengthy evaluation tasks into quick, efficient processes.

• Proven Stability in Real-World Testing

During on-campus testing with 30 students, CodeRoom demonstrated excellent stability. All submissions were processed successfully, the system remained responsive, and the backend–Judge0–MongoDB pipeline handled concurrent execution smoothly.

• Ready for Academic Deployment and Future Growth

With consistent performance, reliable evaluation, and strong user feedback, CodeRoom is ready for real classroom deployment. Its modular architecture allows for future enhancements such as advanced analytics, cloud scaling, and broader language support.

6.3 Future Work

• Plagiarism / Code Similarity Checker

A major enhancement for future versions is the integration of plagiarism detection tools such as MOSS or machine-learning–based similarity models. This feature will allow instructors to automatically compare student submissions and identify copied or highly similar code patterns, ensuring academic honesty during assignments and examinations.

• Advanced Instructor Analytics Dashboard

The analytics dashboard can be expanded with deeper insights that help instructors understand student performance more effectively. Future additions may include topic-wise weaknesses, performance progression over time, frequency of common error types, and predictive charts that forecast student outcomes based on past submissions.

• AI-Based Error Explanations and Hints

Large Language Models (LLMs) can be integrated to provide intelligent feedback to students. The system could automatically generate hints, explain compilation or runtime errors in simple terms, and even suggest logic improvements, helping students learn faster and reducing instructor workload.

• Local Deployment of Judge0

To reduce dependency on external APIs and improve reliability, universities can host a local Judge0 instance through Docker. Running the execution engine on local servers improves performance, reduces latency, and ensures continuous availability even without internet access.

• Support for Additional Programming Languages

The platform can be extended to support more languages such as JavaScript (Node), Go, Kotlin, and SQL. This will make CodeRoom suitable for a wider range of computer science

courses, including backend development, database queries, and modern application programming.

• Classroom Mode with Live Monitoring

A dedicated classroom mode can help instructors manage programming tests more effectively. Features such as real-time submission monitoring, live plagiarism detection, editor locking during exams, and test-specific restrictions can enable fully online proctored programming assessments.

• Intelligent Problem Recommendations for Students

An adaptive recommendation system can suggest assignments based on student performance, difficulty progression, and common error patterns. This personalized approach will help students practice targeted problems and strengthen weaker areas.

• Offline or Weak-Network Mode

To make CodeRoom more accessible in low-connectivity environments, an offline mode can be introduced. Assignments and test cases can be cached locally, and submissions can sync automatically once the network becomes available, improving usability in remote or unstable network regions.

REFERENCES

[1] Docker Documentation, “Docker Engine & Containerization Platform,” 2024. [Online]. Available: https://docs.docker.com/.

[2] Express.js, “Express — Fast, unopinionated, minimalist web framework for Node.js,” 2024. [Online]. Available: https://expressjs.com/.

[3] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2003. (Referenced for backend architecture patterns)

[4] N. S. Gill, “Software Engineering: Systems Development,” Khanna Publishing, 2018. *(General reference used for SDLC, diagrams, and software modeling concepts).*

[5] Judge0 API, “Open-source online code execution system,” 2024. [Online]. Available: https://judge0.com/.

[6] MongoDB Documentation, “MongoDB NoSQL Database,” MongoDB Inc., 2024. [Online]. Available: https://www.mongodb.com/docs/.

[7] Node.js, “Node.js v20 Documentation,” OpenJS Foundation, 2024. [Online]. Available: https://nodejs.org/en/docs.

[8] ReactJS, “React — A JavaScript library for building user interfaces,” Meta Open Source, 2024. [Online]. Available: https://react.dev/.

[9] Stripe, “API Design Guidelines,” 2023. [Online]. Available: https://docs.stripe.com/api. *(Used as reference for API design structure)*

[10] Tailwind CSS, “Tailwind CSS Documentation,” 2024. [Online]. Available: https://tailwindcss.com/docs.

[11] UML.org, “Unified Modeling Language (UML) Specifications,” Object Management Group (OMG), 2024. [Online]. Available: https://www.uml.org/.